Contents

Monotonic Stack — Complete Interview Handbook

Pattern #10 of 28 | DSA Pattern Mastery Series Audience: Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — Stack & Queue section

---

Table of Contents

SECTION 1 — Pattern Overview

1.1 What Is This Pattern?

A Monotonic Stack maintains its elements in strictly increasing or strictly decreasing order at all times. As you scan an array, you **pop** elements from the stack that violate the monotonic property before pushing the current element — and each pop reveals a **"next greater/smaller element"** relationship between the popped element and the current one. This turns an apparent $O(n^2)$ "for each element, scan forward/backward to find the next greater" brute force into O(n).

1.2 Why Was This Pattern Invented?

Finding "the next greater element" for every array position naively requires, for each index, scanning forward until a larger value is found — $O(n^2)$ worst case. The insight: **once you know an element is not the answer for some future index, it can never become relevant again if a nearer, taller candidate appears** — i.e., elements that are "dominated" by a later, better candidate can be permanently discarded. Maintaining only the "undominated so far" elements in a stack, in monotonic order, captures exactly this discard logic in O(n) total.

1.3 Real Intuition Behind The Pattern

Imagine a **line of people of different heights standing single-file, and each person wants to know the height of the first taller person somewhere ahead of them in the line.** As you walk down the line, any time you find someone taller than the people waiting "unanswered" behind you, all of them immediately get their answer (this new taller person), and they can be dismissed — they'll never need to look further, since this is the *first* taller person they encounter. This is exactly the monotonic stack's pop-and-resolve mechanism.

1.4 Mental Model

The stack always holds a monotonic sequence of "candidates still waiting for their answer." A new element either extends the monotonic property (push) or resolves several waiting candidates at once (pop them, record their answer, then push).

1.5 Visual Explanation

```
arr = [2, 1, 2, 4, 3]     find next greater element for each index

i=0(2): stack=[] → push 0 → stack=[0]
i=1(1): 1 < arr[stack.top]=2 → push 1 → stack=[0,1]
i=2(2): 2 > arr[stack.top]=1 → pop 1, next_greater[1]=2;
         2 == arr[stack.top]=2? not strictly greater → push 2 → stack=[0,2]
i=3(4): 4 > arr[2]=2 → pop 2, next_greater[2]=4
         4 > arr[0]=2 → pop 0, next_greater[0]=4
         stack empty → push 3 → stack=[3]
i=4(3): 3 < arr[3]=4 → push 4 → stack=[3,4]

Remaining stack indices [3,4] have no next greater → next_greater[3]=-1, next_greater[4]=-1
Result: [4, 2, 4, -1, -1]
```

1.6 Simple Analogy

Monotonic Stack is like a **stack of pancakes where you only ever add a pancake if it's smaller than the one currently on top** — the moment a bigger pancake comes along, you remove (and "serve" — resolve) every smaller pancake underneath it that's now dominated, before placing the new one down.

1.7 When Should I Immediately Think About Using This Pattern?

- "Next greater/smaller element" (to the left or right).
- "Daily temperatures" style — how many days/steps until a bigger/smaller value.
- "Largest rectangle in histogram" — needing the nearest smaller element on both sides.
- "Trapping rain water" — an alternative monotonic-stack-based solution.
- Any problem where an $O(n^2)$ brute-force "for each element, scan for the nearest X" can be converted to O(n) by tracking "undominated" candidates.

SECTION 2 — Recognition Guide

2.1 Keywords

Keyword	Signal
"next greater element"	Direct signal
"next smaller element"	Direct signal (decreasing monotonic stack)
"daily temperatures," "days until warmer"	Next greater element variant
"largest rectangle in histogram"	Nearest smaller element on both sides
"span," "stock span problem"	Monotonic stack tracking consecutive dominance
"remove k digits to make smallest number"	Monotonic stack for digit removal

2.2 Hidden Hints

A brute-force approach that naturally involves "for each element, look forward/backward until you find one that's bigger/smaller" is the single strongest tell — this exact shape is what Monotonic Stack optimizes from $O(n^2)$ to O(n).

2.3 Interview Clues

Interviewer explicitly asks "can you do this in one pass, O(n)?" after you propose a nested-loop solution for a "next greater element" style problem.

2.4 Common Trick Words

"Span," "next/previous greater/smaller," "visible," "remove to make the smallest/largest" — all point to stack-based monotonic elimination.

2.5 What Interviewers Expect

Correct choice of increasing vs. decreasing stack (matching whether you need "next greater" or "next smaller"), correct handling of "store indices, not values" (needed to compute distances/spans), and amortized O(n) justification despite the nested-looking while-loop.

2.6 When NOT To Use This Pattern

- You need information about **all** pairs, not just nearest greater/smaller — that's a different (often $O(n^2)$ inherent) problem.
- The relevant relationship isn't about **nearest dominance** in a single direction — e.g., you need global max/min, which is a simple O(n) scan, not a stack.
- You need **sliding window** minimum/maximum over a fixed-size window — that's Monotonic **Queue/Deque** (Pattern #11), a close cousin but distinct due to the need to evict from *both* ends.

SECTION 3 — Decision Framework

```
Does the problem ask for "next/previous greater/smaller element" for every position?

       Yes → USE MONOTONIC STACK

        No

Does it ask for MIN/MAX within a SLIDING WINDOW of fixed size?

       Yes → USE MONOTONIC QUEUE/DEQUE (Pattern #11) instead - need eviction from both ends

        No

Does it need matching pairs (e.g., valid parentheses, nesting structure)?

       Yes → USE A REGULAR STACK (LIFO matching), not necessarily monotonic

        No

Does it need GLOBAL max/min only (not nearest-per-position)?

       Yes → Simple O(n) linear scan suffices - no stack needed at all
```

Why: Monotonic Stack's specific value is answering "nearest greater/smaller in one direction, for every position, in one pass" — a narrower but very common question shape. Confusing it with sliding-window min/max (needs eviction from the window's back, requiring a deque) or with simple global extremes (needs no stack at all) is a common pattern-misapplication mistake.

SECTION 4 — Why This Pattern Works

Mathematical/Logical: Each element is pushed onto the stack exactly once and popped at most once — total operations across the entire algorithm are bounded by $2n = O(n)$, even though the while-loop inside the main loop looks like it could be expensive per iteration. This is the same amortized argument as Two Pointers' duplicate-skipping loops: **the total work across all inner-loop iterations, summed over the entire outer loop, is bounded by n, not n^2**, because each element can only be popped once.

Intuitive: Once a taller/bigger element appears, every shorter/smaller element still "waiting" in the stack immediately gets its answer resolved and is permanently done — there's no reason to ever reconsider it, since the current element is the *first* qualifying one for all of them simultaneously (they were all waiting in increasing order of "distance already passed without finding an answer").

Correctness Proof: *Invariant:* at any point during the scan, the stack contains indices of elements that (a) have not yet found their "next greater" element, and (b) are stored in strictly increasing order of their values (for a "next greater" search), meaning the stack itself is monotonically decreasing from bottom to top... more precisely: the stack values are in increasing order from top to bottom is not quite right — the exact invariant is: **the stack holds indices whose values form a strictly decreasing sequence from bottom to top, i.e., each new candidate is smaller than the one below it, representing elements still awaiting a bigger element to their right.** *Base case:* stack starts empty — trivially satisfies the invariant. *Inductive step:* when a new element arrives, popping every stack element smaller than it (resolving their "next greater"

answer) and then pushing the new element preserves the strictly-decreasing-from-bottom invariant. *Termination:* after the full scan, any indices remaining on the stack have no "next greater" element (answer = -1 or sentinel), which is correctly the case since nothing after them in the array exceeded their value. **QED.**

SECTION 5 — Algorithm Framework

5.1 Step-by-Step Framework (Next Greater Element, left-to-right)

1. Initialize an empty stack (storing indices) and a result array filled with a sentinel (e.g., -1).
2. For each index `i` from 0 to n-1: while the stack is non-empty and `arr[i] > arr[stack.top]`, pop the top index `j` and set `result[j] = arr[i]`.
3. Push `i` onto the stack.
4. After the loop, any indices remaining on the stack have no next greater element (result stays at sentinel).

5.2 General Template

```
function nextGreaterElement(arr):
    n = length(arr)
    result = array of size n, filled with -1
    stack = []                      # stores indices
    for i in range(0, n):
        while stack is not empty and arr[i] > arr[stack.top()]:
            j = stack.pop()
            result[j] = arr[i]
        stack.push(i)
    return result
```

5.3 Largest Rectangle in Histogram Template (Nearest Smaller Both Sides)

```
function largestRectangleArea(heights):
    stack = []                      # increasing stack of indices
    maxArea = 0
    n = length(heights)
    for i in range(0, n + 1):
        currentHeight = (i == n) ? 0 : heights[i]     # sentinel to flush remaining stack
        while stack is not empty and currentHeight < heights[stack.top()]:
            height = heights[stack.pop()]
            width = stack.isEmpty() ? i : i - stack.top() - 1
            maxArea = max(maxArea, height * width)
        stack.push(i)
    return maxArea
```

5.4 Interview Thinking Process

1. "This needs 'next/previous greater/smaller' for every position — I'll use a monotonic stack instead of a brute-force nested scan."
2. "I'll store indices, not values, on the stack, since I often need positional distance (span, width) not just the value."
3. "I'll decide increasing vs. decreasing stack based on whether I need the next *greater* (decreasing stack, pop on bigger arrival) or next *smaller* (increasing stack, pop on smaller arrival)."
4. "Total work is O(n) amortized, since each element is pushed once and popped at most once."

SECTION 6 — Time & Space Complexity

Case	Time	Space	Why
Worst Case	O(n)	O(n)	Each element pushed once, popped at most once — stack size bounded by n
Average Case	O(n)	O(n)	Same regardless of data distribution
Best Case	O(n) (still must scan once)	O(n) (worst case, e.g., strictly increasing input never pops)	Even a "no pops" scenario requires the full scan
Amortized	O(n) despite the inner while-loop	O(n)	Total pushes + pops across the entire run bounded by 2n

---

SECTION 7 — Edge Cases

Edge Case	Example	Handling
Empty array	`[]`	Return empty result immediately
Single element	`[5]`	No next greater/smaller exists — result is sentinel (-1)
Strictly increasing array	`[1,2,3,4]`	Every element (except last) finds its next greater immediately; stack never grows beyond depth 1 in practice for "next greater" here — actually pops resolve continuously
Strictly decreasing array	`[4,3,2,1]`	No pops ever occur (nothing is ever bigger than the top) — stack grows to full size, all results stay -1
All identical elements	`[3,3,3]`	Must clarify strict (>) vs non-strict (>=) comparison — affects whether equal elements resolve each other
Circular array variant	"Next Greater Element II" (wraps around)	Requires scanning the array twice (conceptually) or using modulo indexing to simulate circularity
Duplicate values requiring stable/specific tie-breaking	Various "span" problems	Clarify whether equal values count as "greater than or equal" per problem statement

Common mistakes: using >= instead of > (or vice versa) inconsistently with what the problem

asks, causing off-by-one errors on ties; forgetting to handle the circular-array variant by not doubling the effective scan range.

SECTION 8 — Pros & Cons

Advantages: Converts $O(n^2)$ nearest-greater/smaller brute force into O(n); conceptually elegant once the "resolve and discard" mental model clicks; broadly applicable to a family of "nearest boundary" problems. **Disadvantages:** Requires storing indices (not just values) for many variants, adding a small layer of indirection that can confuse beginners; strict vs. non-strict comparison choice is a frequent source of subtle bugs. **Trade-offs:** Monotonic Stack (O(n) time, O(n) space) vs. brute force ($O(n^2)$ time, O(1) space) — always prefer the stack when the "nearest greater/smaller for every position" shape is present. **Limitations:** Doesn't directly solve sliding-window min/max (needs Monotonic Deque instead, since elements must be evicted from both ends as the window slides); doesn't answer "all pairs," only "nearest in one direction." **Inefficient when:** N/A for its exact use case — O(n) is optimal since every element must be examined at least once.

SECTION 9 — Real World Applications

Domain	Application
Google	Stock price analysis — "stock span problem" (consecutive days with price ≤ today's price) is a canonical monotonic stack application
Amazon	Warehouse shelf-height optimization (largest rectangular storage area in an irregular shelf layout — histogram problem analog)
Meta	Real-time trending detection — "how many consecutive time buckets had lower engagement than now"
Financial Trading Systems	Computing "days until price exceeds current" for technical indicators (resistance/support level detection)
Compilers	Expression parsing and operator precedence evaluation often uses stack-based nearest-higher-precedence logic
Computational Geometry	Largest rectangle/skyline-area problems in building layout and image processing
Weather/Climate Systems	"Daily Temperatures" style problems directly model "days until warmer weather" forecasting utilities
Networking	Detecting the next higher-priority packet in a priority-ordered buffer using monotonic stack logic
Game Development	Visibility determination (which objects are visible given heights/occlusion) uses monotonic stack-like nearest-taller logic

---

SECTION 10 — Interview Strategy

How seniors answer: They immediately recognize the "next greater/smaller for every position" shape, state the O(n) amortized argument explicitly ("each element is pushed once and popped at most once"), and clarify strict vs. non-strict comparison semantics before coding.

How juniors answer: They often default to an $O(n^2)$ nested loop, or apply a monotonic stack without being able to explain why it's O(n) despite the nested-looking while-loop, leaving them unable to defend correctness under a "why is this efficient?" follow-up.

Typical follow-ups: "What if the array is circular (wraps around)?" "Can you find the answer looking both left and right simultaneously?" "How would you extend this to 2D (largest rectangle in a binary matrix)?"

Optimization questions: "Can you avoid the O(n) space for the stack?" (Generally no — the stack is fundamental to the technique; discuss why it's necessary and already optimal.)

SECTION 11 — Pattern Variations

Variation	Description	Example
Next Greater Element (right)	Decreasing stack, resolve on bigger arrival	Next Greater Element I
Next Smaller Element	Increasing stack, resolve on smaller arrival	Next Smaller Element (variant)
Previous Greater/Smaller	Scan right-to-left or maintain stack differently	Online Stock Span
Circular Array Variant	Simulate wraparound via doubled iteration or modulo	Next Greater Element II
Nearest Smaller Both Sides	Combine left-scan and right-scan stacks	Largest Rectangle in Histogram
Digit Removal / Monotonic Construction	Build the smallest/largest result by popping violating digits	Remove K Digits, Remove Duplicate Letters
Trapping Rain Water (stack variant)	Nearest taller wall on both sides via stack	Trapping Rain Water

---

SECTION 12 — Comparison With Other Patterns

Pattern	Difference	When To Prefer
Monotonic Queue/Deque	Supports eviction from both ends for sliding-window min/max	Fixed-size window min/max, not single-direction nearest search
Two Pointers	Searches pairs/triplets via elimination, not nearest-dominance relationships	Pair/triplet search on sorted data
Regular Stack (LIFO matching)	Used for nested/matching structure (parentheses), not dominance	Bracket matching, nested structure validation

Pattern	Difference	When To Prefer
Dynamic Programming	Some “largest rectangle” style problems could be brute-forced with DP, but monotonic stack is strictly faster for this specific shape	When no clean “nearest dominance” structure exists

Comparison Table

Aspect	Monotonic Stack	Monotonic Queue/Deque	Regular Stack
Eviction direction	One end (top) only	Both ends	One end (top) only
Use case	Nearest greater/smaller (one direction)	Sliding window min/max	Matching/nesting (LIFO)
Time	O(n)	O(n)	O(n)

SECTION 13 — Problem Classification

Difficulty	Characteristics	Examples
Easy	Direct next-greater on a simple array	Next Greater Element I
Medium	Circular arrays, span calculations, digit removal	Next Greater Element II, Online Stock Span, Remove K Digits, Daily Temperatures
Hard	Nearest smaller both sides, histogram-based area	Largest Rectangle in Histogram, Trapping Rain Water (stack variant)
Very Hard	2D extensions, combined with DP or advanced counting	Maximal Rectangle (2D histogram extension), Sum of Subarray Minimums at scale

---

SECTION 14 — 30 Interview Problems

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
1	Next Greater Element I	Easy	Amazon, Meta	Direct decreasing monotonic stack	Foundational mechanics
2	Next Greater Element II	Medium	Amazon, Google	Circular array handling	Wraparound simulation

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
3	Daily Temperatures	Medium	Amazon, Meta, Google, Microsoft	Next greater element (distance, not value)	Distance-based variant
4	Online Stock Span	Medium	Amazon, Google	Previous greater element (span counting)	Span calculation via stack
5	Largest Rectangle in Histogram	Hard	Amazon, Meta, Google, Microsoft	Nearest smaller both sides	Advanced area calculation
6	Maximal Rectangle	Hard	Amazon, Google, Meta	2D extension of histogram problem	Dimensional extension
7	Trapping Rain Water	Hard	Amazon, Google, Meta, Uber	Nearest taller wall via stack (alternative to Two Pointers)	Cross-pattern contrast
8	Remove K Digits	Medium	Amazon, Google	Monotonic stack for smallest number construction	Constructive monotonic stack
9	Remove Duplicate Letters	Medium	Google, Amazon	Monotonic stack with last-occurrence constraint	Constrained constructive stack
10	Sum of Subarray Minimums	Medium	Amazon, Google	Nearest smaller both sides + contribution counting	Advanced contribution technique
11	Sum of Subarray Ranges	Medium	Google, Amazon	Combines min and max monotonic stack contributions	Dual monotonic stack combination
12	132 Pattern	Medium	Google, Amazon	Monotonic stack for pattern detection	Pattern-matching via stack
13	Asteroid Collision	Medium	Amazon, Google, Meta	Stack-based collision simulation	Simulation via stack
14	Car Fleet	Medium	Google, Amazon	Monotonic stack for fleet merging	Simulation + monotonicity

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
15	Score of Parentheses	Medium	Amazon, Google	Stack-based nested scoring	Related stack application (not strictly monotonic)
16	Basic Calculator	Hard	Amazon, Google, Meta	Stack-based expression evaluation	Related stack application
17	Valid Parenthesis String	Medium	Amazon, Google	Related stack/greedy hybrid	Contrast with monotonic stack
18	Decode String	Medium	Amazon, Meta, Google	Stack-based nested decoding	Related stack application
19	Simplify Path	Medium	Amazon, Meta	Stack-based path resolution	Related stack application
20	Exclusive Time of Functions	Medium	Amazon, Google	Stack-based interval tracking	Related stack application
21	Minimum Remove to Make Valid Parentheses	Medium	Meta, Amazon	Stack-based validation and removal	Related stack application
22	Maximum Width Ramp	Medium	Google, Amazon	Monotonic stack for ramp-width maximization	Advanced monotonic stack variant
23	Shortest Unsorted Continuous Subarray	Medium	Amazon, Google	Related monotonic boundary detection	Boundary detection via monotonicity
24	Next Greater Node In Linked List	Medium	Amazon, Google	Monotonic stack applied to linked list traversal	Cross-pattern (Linked List + Stack)
25	Remove Nodes From Linked List	Medium	Google, Amazon	Monotonic stack applied to linked list construction	Cross-pattern (Linked List + Stack)
26	Final Prices With a Special Discount in a Shop	Easy	Amazon	Direct next-smaller-element application	Basic application variant
27	Number of Visible People in a Queue	Hard	Google, Amazon	Advanced monotonic stack with visibility counting	Advanced visibility logic

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
28	Longest Well-Performing Interval	Medium	Google	Related prefix-sum + hashmap (contrast, not pure monotonic stack)	Pattern-boundary awareness
29	Build Array Where You Can Find The Maximum Exactly K Comparisons (contrast)	Hard	Google	Contrast: DP, not monotonic stack	Pattern-boundary awareness
30	Minimum Cost Tree From Leaf Values	Medium	Google, Amazon	Monotonic stack for optimal tree construction	Advanced constructive application

---

SECTION 15 — Common Mistakes

1. Storing values instead of indices when distances/spans/widths are needed. *Fix:* default to storing indices; dereference `arr[index]` when the value is needed.
2. Inconsistent strict (>) vs non-strict (>=) comparison relative to what the problem requires for ties. *Fix:* explicitly clarify and test with a tie-containing example.
3. Forgetting to handle circular arrays by not doubling the effective iteration (via modulo indexing or concatenation). *Fix:* explicitly simulate wraparound with `i % n` over a `2n` iteration range.
4. Misapplying Monotonic Stack to sliding-window min/max problems, where eviction from *both* ends is needed — leads to incorrect logic since a simple stack can't evict from the bottom. *Fix:* recognize this needs Monotonic Deque (Pattern #11) instead.
5. Forgetting the "sentinel" flush step (e.g., a height of 0 appended at the end) in histogram-style problems, leaving some stack elements unresolved. *Fix:* always add a sentinel pass to flush the remaining stack at the end.

Why people fail: the "aha" insight (elements get permanently resolved and discarded) is non-obvious the first time, and candidates who haven't internalized *why* it's O(n) often either avoid the technique entirely (falling back to $O(n^2)$) or apply it superficially without correctly identifying whether they need indices or values, or increasing or decreasing order.

SECTION 16 — Optimization Techniques

- **Time:** Already optimal at O(n) — focus on avoiding redundant extra passes (e.g., combine left-scan and right-scan into a single traversal with two stacks where possible).
- **Space:** Reuse the result array as auxiliary storage where possible to avoid extra allocations; for very large inputs, consider whether values can be processed in a streaming fashion.
- **Readability:** Name the stack by its semantic role (`decreasingStack`, `candidateIndices`), and comment the invariant explicitly ("stack holds indices with strictly decreasing values, awaiting next greater").

- **Interview performance:** Explicitly state the amortized O(n) argument before coding — this preempts the most common correctness/efficiency follow-up question.

SECTION 17 — Multiple Language Templates

Java

```java
public int[] nextGreaterElement(int[] arr) {
    int n = arr.length;
    int[] result = new int[n];
    Arrays.fill(result, -1);
    Deque<Integer> stack = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        while (!stack.isEmpty() && arr[i] > arr[stack.peek()]) {
            result[stack.pop()] = arr[i];
        }
        stack.push(i);
    }
    return result;
}
```

JavaScript

```javascript
function nextGreaterElement(arr) {
    const n = arr.length;
    const result = new Array(n).fill(-1);
    const stack = [];
    for (let i = 0; i < n; i++) {
        while (stack.length && arr[i] > arr[stack[stack.length-1]]) {
            result[stack.pop()] = arr[i];
        }
        stack.push(i);
    }
    return result;
}
```

PHP

```php
function nextGreaterElement(array $arr): array {
    $n = count($arr);
    $result = array_fill(0, $n, -1);
    $stack = [];
    for ($i = 0; $i < $n; $i++) {
        while (!empty($stack) && $arr[$i] > $arr[end($stack)]) {
            $result[array_pop($stack)] = $arr[$i];
        }
        $stack[] = $i;
    }
    return $result;
}
```

Python

```python
def next_greater_element(arr):
    n = len(arr)
    result = [-1] * n
    stack = []
    for i in range(n):
        while stack and arr[i] > arr[stack[-1]]:
            result[stack.pop()] = arr[i]
        stack.append(i)
    return result
```

Go

```go
func nextGreaterElement(arr []int) []int {
    n := len(arr)
    result := make([]int, n)
    for i := range result {
        result[i] = -1
    }
    stack := []int{}
    for i := 0; i < n; i++ {
        for len(stack) > 0 && arr[i] > arr[stack[len(stack)-1]] {
            top := stack[len(stack)-1]
            stack = stack[:len(stack)-1]
            result[top] = arr[i]
        }
        stack = append(stack, i)
    }
    return result
}
```

C++

```cpp
vector<int> nextGreaterElement(vector<int>& arr) {
    int n = arr.size();
    vector<int> result(n, -1);
    stack<int> st;
    for (int i = 0; i < n; i++) {
        while (!st.empty() && arr[i] > arr[st.top()]) {
            result[st.top()] = arr[i];
            st.pop();
        }
        st.push(i);
    }
    return result;
}
```

SECTION 18 — Dry Runs

Small Input

```
arr = [2, 1, 2, 4, 3]
```

```
i=0(2): stack=[] → push → stack=[0]
i=1(1): 1 < arr[0]=2 → push → stack=[0,1]
i=2(2): 2 > arr[1]=1 → pop 1, result[1]=2; 2 > arr[0]=2? No (equal) → push → stack=[0,2]
i=3(4): 4 > arr[2]=2 → pop 2, result[2]=4; 4 > arr[0]=2 → pop 0, result[0]=4; push → stack=[3]
i=4(3): 3 < arr[3]=4 → push → stack=[3,4]
Remaining stack [3,4] → result[3]=-1, result[4]=-1
Final: [4, 2, 4, -1, -1]
```

Large Input (Conceptual)

For an array of 10^6 elements, total pushes = 10^6 and total pops ≤ 10^6, so total stack operations ≤ 2×10^6 — confirming O(n) regardless of how "clustered" the greater-element resolutions are.

Corner Case

`arr = [5,4,3,2,1]` (strictly decreasing): every push happens, no pops ever occur (nothing is ever bigger than what's below it) — final stack contains all 5 indices, all results remain -1, correctly reflecting that no element has a next greater element.

SECTION 19 — Advanced Concepts

- **Contribution technique (Sum of Subarray Minimums):** instead of computing the minimum of every subarray directly ($O(n^2)$ or worse), use a monotonic stack to find, for each element, how many subarrays it is the *minimum* of (via its nearest smaller elements on both sides), then sum `value × count` — a powerful generalization from "next greater/smaller" to "counting contribution across all subranges."
- **Dual monotonic stacks:** some problems (Sum of Subarray Ranges) require both a "nearest smaller" stack (for minimums) and a "nearest greater" stack (for maximums) run independently, then combined.
- **Monotonic stack for constructive/greedy problems (Remove K Digits, Remove Duplicate Letters):** here the stack isn't just answering a query — it's actively building the final answer, popping digits/characters that violate the target monotonic order as long as removal budget/constraints allow.
- **Circular array simulation:** conceptually iterate `2n` times using `i % n` indexing (or physically duplicate the array) to correctly resolve wraparound "next greater" relationships without literally doubling memory usage where avoidable.

SECTION 20 — Senior Engineer Insights

Staff engineers recognize the Monotonic Stack pattern as a specific instance of a broader amortized-analysis principle: **"each element does a bounded amount of total work across the entire algorithm, even though any single step might look expensive."** This same amortized reasoning underlies dynamic array resizing, union-find path compression, and splay tree operations. Interviewers evaluate whether a candidate can articulate this amortized argument rigorously (not just "trust me, it's O(n)") and whether they can extend the basic "next greater element" template to constructive problems (build the optimal result via monotonic popping) — a noticeably harder skill than simple query-answering.

SECTION 21 — Cheat Sheet

```
PATTERN: Monotonic Stack
RECOGNIZE: "next/previous greater/smaller element," "daily temperatures," "largest rectangle in histogram
TEMPLATE (next greater, decreasing stack):
    stack = []   # holds indices, decreasing values bottom to top
    for i in range(n):
        while stack and arr[i] > arr[stack.top]: result[stack.pop()] = arr[i]
        stack.push(i)
COMPLEXITY: O(n) time, O(n) space
KEY PROOF: each element pushed once, popped at most once - amortized O(n) total
WATCH FOR: index vs value storage, strict vs non-strict comparison, circular array handling, sentinel flus
DOESN'T APPLY WHEN: need sliding-window min/max (use Monotonic Deque), need all-pairs relationships
```

SECTION 22 — Revision Notes (5-Minute Review)

- Monotonic Stack resolves "next/previous greater/smaller for every position" in O(n) amortized.
- Store indices (not values) when distance/span/width matters.
- Decreasing stack → resolves "next greater"; increasing stack → resolves "next smaller."
- Each element pushed once, popped at most once — that's the amortized O(n) proof.
- Sliding window min/max needs Monotonic Deque (both-end eviction), not a plain stack.
- Can be used constructively (Remove K Digits) to build optimal results, not just answer queries.

SECTION 23 — Practice Roadmap

Stage	Focus	Recommended LeetCode Problems
Beginner	Basic next-greater mechanics	Next Greater Element I (496), Daily Temperatures (739)
Intermediate	Circular arrays, span, constructive stacks	Next Greater Element II (503), Online Stock Span (901), Remove K Digits (402)
Advanced	Nearest smaller both sides, area calculation	Largest Rectangle in Histogram (84), Sum of Subarray Minimums (907)
Expert	2D extension, dual-stack combination	Maximal Rectangle (85), Sum of Subarray Ranges (2104), Trapping Rain Water (42, stack variant)

SECTION 24 — Memory Tricks

- **Mnemonic:** "**P**op, **R**esolve, **P**ush" (PRP) — Pop dominated elements, Resolve their answer, Push the new one.
- **Visualization:** A **line of people waiting for the next taller person** — the moment someone taller arrives, everyone shorter behind them gets their answer and steps out of line.

- **Recognition shortcut:** “Next/previous greater/smaller” for every element → Monotonic Stack, decide increasing vs decreasing based on which direction you need.

SECTION 25 — Final Summary

Monotonic Stack converts the brute-force $O(n^2)$ search for “next/previous greater or smaller element” into O(n) amortized time by maintaining a stack of “undominated” candidates and resolving several of them at once whenever a new dominating element arrives. The single most important thing to remember forever: **every element is pushed exactly once and popped at most once — that single fact is the entire amortized O(n) proof — and the stack should store indices, not values, whenever distance, width, or span is part of the answer.** When the question shifts from “one-directional nearest dominance” to “sliding-window extremes,” pivot immediately to Monotonic Queue/Deque instead.